Université Paris Cité

Master 1 – Computer Science

EventHub - Neurovent

Scientific and Tech Event Management Platform

Project Report

Full-stack Architecture: React / Django / Node.js

Students: GOURMELEN Thomas *(Student A)*
MOHAMMEDI Noureddine *(Student B)*
ZOUAOUI Azouaou *(Student C)*

Course: *Web Programming*
Instructor: *ALLA Jammine*

Application (Django): `neurovent-web.vercel.app`
Application (Node.js): `neuro-vent-mu.vercel.app`

Date: April 10, 2026

Contents

1 Introduction and System Overview

1.1 Context and Problem Statement

Observation. Organising scientific and technical events requires coordination between multiple stakeholders: organisers, participants, and platform administrators.

Limitations of manual management. Without a dedicated tool, manual management (emails, spreadsheets) leads to registration errors, a lack of real-time tracking, and an inability to handle waitlists.

Neurovent's response. Neurovent addresses this need by providing a centralised platform covering the entire lifecycle of an event: creation, publication, registrations, moderation, and post-event analysis.

Chosen architecture. The application relies on a three-layer full-stack architecture: a main backend in **Django REST Framework**, a frontend in **React**, and a secondary backend in **Node.js/Express** developed for technology comparison purposes.

1.2 What Neurovent Does

Neurovent is built around three distinct roles, each with a dedicated functional space:

- **Participant**: event discovery and filtering, registration (immediate or pending validation), profile management, and tracking of personal registrations.
- **Organisation**: event creation and publication, registration moderation, access to an analytics dashboard, and CSV exports.
- **Administrator**: global platform supervision, account moderation, organisation verification via the SIRENE API, and cross-platform statistics.

Common ground. All three roles share the same user base (`CustomUser`) and interact with the same business entities, with access levels controlled by JWT and granular API-level permissions.

1.3 Compliance with the Project Requirements

TABLE 1. Coverage of the project requirements.

Project Requirement	Neurovent Feature
Event management (CRUD)	Creation, editing, deletion, and listing with 6 filter criteria
Participant management (CRUD)	Full profile, search, suspension, and admin moderation
Registration (Many-to-Many)	`Registration` model with statuses, uniqueness constraint, and automatic waitlist management
Authentication (min. 2 roles)	JWT with blacklist, 3 roles: `PARTICIPANT`, `COMPANY`, `ADMIN`
Event filtering	6 criteria on both API and frontend (format, tags, dates, city, country, text search)
Deployment	Backend and frontend deployed, environment variables configured, application publicly accessible

Report structure. The remainder of the report covers the general system architecture, the Django backend and its business rules, the React frontend and its API integration, the Node.js backend and the technology comparison with Django, and finally the production deployment.

1.4 Visual Overview

Screenshots of the deployed application are collected in Appendix D. They cover the three main user flows: the participant experience (event search, results listing, and event detail), the organisation space (dashboard, event creation, registration management, and company profile), the administrator interface (organisation list and pending verification workflow), and a mobile responsive view (Figure 15).

2 General Architecture

2.1 Project Structure

Organisation. The project is structured as a monorepo with three independent components:

```
Projet/
 backend-django/ # Main API (Django REST Framework)
 backend-node/ # Comparative API (Node.js / Express)
 frontend-react/ # User interface (React SPA)
```

Autonomy principle. Each component is self-contained, with its own dependencies and its own local SQLite database. Figure 1 illustrates the interactions between these components.

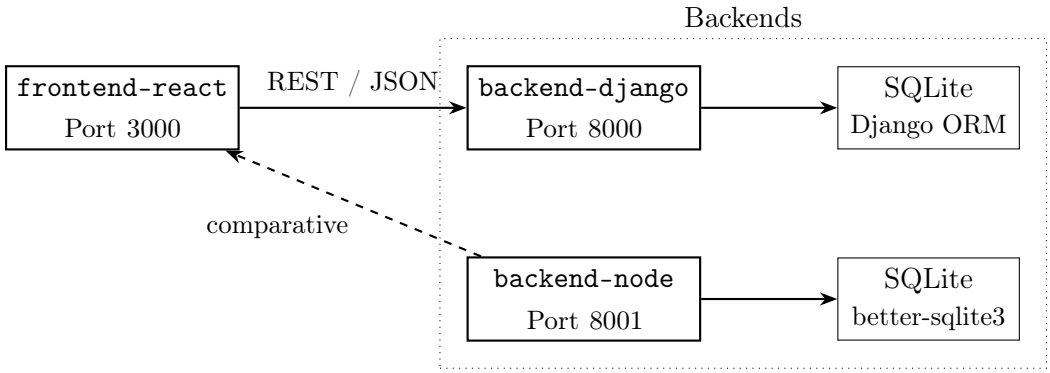

FIGURE 1. General architecture of the Neurovent project (monorepo).

Role distribution. The frontend communicates exclusively with the Django backend via a REST/JSON API. The Node.js backend exposes the same business entities on a separate port and is used solely for technology comparison purposes; it does not serve data to the interface in production.

2.2 Communication Flow and Authentication

General flow. All requests between the React frontend and the Django backend transit over HTTP. Upon login, the backend returns two JWT tokens: an access token (2-hour lifetime) and a refresh token (7 days), both stored in the browser's `localStorage`. Each authenticated request includes the access token in the `Authorization: Bearer <token>` HTTP header. A frontend interceptor (`apiFetch`) automatically detects `401` responses and attempts a silent renewal via the refresh token before retrying the original request. This mechanism ensures a transparent session for the user without forcing them to log in every two hours.

CORS. The Django backend's CORS configuration explicitly allows the frontend origins (via the `CORS_ALLOWED_ORIGINS` variable), a necessary condition for cross-origin requests to work in both development and production.

2.3 Justified Technology Choices

TABLE 2. Justification of technology choices.

Technology	Role	Justification
Django + DRF	Main backend	Mature ORM, built-in serialisation, declarative permissions, Django admin
Node.js / Express	Comparative backend	Lightweight architecture, explicit controllers, empirical comparison baseline
React (SPA)	Frontend	Client-side routing, reusable components, rich ecosystem
SQLite	Persistence (dev)	Zero configuration, portability, replaceable by PostgreSQL in production
JWT (Simple-JWT)	Authentication	Stateless, short-lived access token (2h) + refresh token (7 days)

3 Django Backend

3.1 Data Models

3.1.1 `CustomUser` – Extended User Model

Model role. The user model replaces Django's default `User` by extending `AbstractBaseUser` and `PermissionsMixin`. It centralises the identity of all platform actors through a discriminant `role` field (`PARTICIPANT`, `COMPANY`, `ADMIN`), avoiding multiple user tables while still enabling heterogeneous profiles.

Authentication choices. The two login types are deliberately distinct:

- participants authenticate via **email**;
- organisations use a **text identifier** (`company_identifier`) subject to regex validation.

This separation reflects different use cases and avoids any namespace collision.

Both profiles coexist in the same table via `null=True, blank=True` fields conditioned on the role:

- **Participant profile**: avatar, biography, profile type (`STUDENT` or `PROFESSIONAL`), study level or job title, favourite domain, GitHub, LinkedIn, and personal website links.
- **Organisation profile**: logo, description, SIRET number, legal representative, verification status (`PENDING`, `VERIFIED`, `REJECTED`, `NEEDS_REVIEW`), and social links (YouTube, LinkedIn, Twitter, Instagram, Facebook).

3.1.2 `Event` – Rich Event Model

Functional scope. The `Event` model covers the entire lifecycle of an event.

Core elements.

- **Persisted statuses**: `DRAFT`, `PUBLISHED`, and `CANCELLED`.
- **Temporality**: whether an event is past or ongoing is determined dynamically from `date_start` and `date_end`, without any additional status field.
- **Formats**: `ONSITE`, `ONLINE`, or `HYBRID`.
- **Registration modes**: `AUTO` or `VALIDATION`.

Two computed properties complement the model:

- `registration_open`: accounts for the deadline, the event end date, and the `allow_registration_during_event` flag.
- `spots_remaining`: returns capacity minus registrations with `CONFIRMED` status.

Deferred visibility. A deferred visibility feature allows the physical address and online link to be hidden until a configurable date (`address_reveal_date`, `online_reveal_date`). Before that date, only the city and country, or the platform name, are exposed.

Traceability. Eight timestamped fields track the sending of reminders and alerts (D-7, D-1, H-3, capacity alerts, reveals), ensuring idempotency of scheduled tasks.

3.1.3 `Registration` – The Central Many-to-Many Relationship

Central role. The `Registration` model materialises the Many-to-Many relationship between `CustomUser` and `Event`. It is not limited to a simple join table: it embeds full business logic with five possible statuses — `PENDING`, `CONFIRMED`, `REJECTED`, `CANCELLED`, and `WAITLIST` — each driving a distinct behaviour in the registration workflow.

Business guarantees.

- The `unique_together(participant, event)` constraint enforces uniqueness at the database level.
- It is complemented by serializer-level validation that detects any existing active registration before any write operation, returning an explicit business error.
- Two fields enrich each registration: `accessibility_needs` for the participant's specific requirements, and `company_comment` for the organiser's note on approval or rejection.

Waitlist management relies on the computed property `waitlist_position`, which dynamically determines each participant's rank:

```
@property
def waitlist_position(self):
    """Position in the waitlist (1 = first). None if not waiting."""
    if self.status != RegistrationStatus.WAITLIST:
        return None
    return (
        Registration.objects
        .filter(
            event=self.event,
            status=RegistrationStatus.WAITLIST,
            created_at__lt=self.created_at
        )
        .count() + 1
    )
```

3.1.4 `Tag` – Taxonomy and Recommendations

Tags are associated with events and participant profiles via Many-to-Many relationships. A recommendation engine exploits the intersection of tags between a participant's profile and published events to suggest relevant content.

3.2 REST API with Django REST Framework

3.2.1 Authentication and Permissions

JWT mechanism.

- Authentication relies on **SimpleJWT** with a 2-hour access token and a 7-day refresh token.
- On logout, the refresh token is server-side blacklisted, enabling immediate revocation without shared state.
- The token payload carries custom claims — role, `company_name`, and `company_identifier` — allowing the frontend to adapt its interface at login time, without an additional `/me` call.

Access control. Permission granularity is ensured by custom permission classes, `IsCompany`, `IsParticipant`, and `IsCompanyOwner`, applied directly at each view level.

Authentication endpoint coverage.

- registration and login, each on two separate routes (participant / organisation);
- profile retrieval and modification via `/me` (GET, PATCH, DELETE);
- logout and two-step password reset (request then token confirmation).

3.2.2 Serializers and Error Handling

Principle. Each entity has several specialised serializers depending on context.

- For events, `EventListSerializer` exposes the fields needed for list display, while `EventDetailSerializer` enriches the response with registration data and computed properties.
- For users, the exposed fields vary depending on whether the requester is the profile owner, another participant, or an administrator.
- This separation prevents any sensitive data leakage and reduces response payload size.

Error handling follows HTTP conventions:

- `400` for business validation errors (duplicate registration, expired deadline, capacity reached);
- `401` for unauthenticated requests;
- `403` for unauthorised access (wrong role, resource belonging to another user);
- `404` via `get_object_or_404` for missing resources.

Response format. Error messages are returned as JSON with a `detail` key or per-field keys, directly usable by the frontend.

3.2.3 Event CRUD

Table 3. Event endpoints – summary.

Endpoint	Access	Description
`GET /events/`	Public	List with filters (format, tags, dates, city, country, text)
`GET /events/:id/`	Public	Event detail
`POST /events/create/`	Organisation	Create an event
`PATCH /events/:id/update/`	Owner	Partial update
`DELETE /events/:id/delete/`	Owner	Delete
`GET /events/my-events/`	Organisation	Organisation's own events
`GET /events/:id/stats/`	Owner	Registration statistics
`GET /events/dashboard-stats/`	Organisation	Global summary + performance
`GET /events/dashboard-stats/export-*`	Organisation	CSV export of registrations

Available filters. Filters are implemented via **django-filter**:

- format;
- tags (OR logic);
- `date_after` / `date_before`;
- city;
- country;
- free text search;
- configurable `ordering`.

3.2.4 Registration Management

Validation sequence. On registration submission, several checks are chained:

- event has `PUBLISHED` status;
- deadline is not exceeded;
- `registration_open` property is true;
- no existing active registration found.

The final status is then determined based on remaining capacity and the registration mode:

```
confirmed_count = event.registrations.filter(
    status=RegistrationStatus.CONFIRMED
).count()
is_full = False if event.unlimited_capacity else (
    confirmed_count >= event.capacity
)

if is_full:
    if event.registration_mode == 'AUTO':
        new_status = RegistrationStatus.WAITLIST
    else:
        raise ValidationError("This event is full.")
else:
    new_status = (
        RegistrationStatus.CONFIRMED
        if event.registration_mode == 'AUTO'
        else RegistrationStatus.PENDING
    )
```

Business behaviour.

- In `AUTO` mode, an available spot triggers immediate confirmation.
- In `VALIDATION` mode, the registration remains pending manual approval.
- If the event is full, the participant joins the waitlist.
- On cancellation or rejection, the `_promote_from_waitlist` function automatically promotes the first `WAITLIST` entry and sends them a confirmation email.

Organisation moderation. The organisation has dedicated moderation endpoints: approval, rejection, removal, and CSV export of the registrant list.

3.2.5 Django Admin

Admin configuration.

- Each application has its own `admin.py` configuration.
- `CustomUserAdmin` extends `UserAdmin` and organises participant and organisation fields into distinct fieldsets, with role/status filters and multi-field search.
- `EventAdmin` structures fields into thematic sections (basic info, format and registration, location, online link), with filters by status, format, and registration mode.

This configuration enables full platform administration directly from the Django interface, without additional frontend development.

3.2.6 Emails, Documentation, and Management Commands

Transactional emails. The backend handles several types of transactional emails:

- registration confirmation;
- waitlist notification;
- pre-event reminders (D-7, D-1, H-3);
- address or link reveal;
- organiser digest.

An `email-previews/` folder enables local HTML preview during development.

Documentation and automation.

- API documentation is automatically generated via **drf-spectacular** (Swagger UI and ReDoc).
- Dedicated `manage.py` commands allow reminders and notifications to be triggered idempotently.

3.3 Production Configuration

Production stack.

- **Gunicorn** acts as the WSGI server.
- **Whitenoise** handles static files without an intermediate web server.
- **psycopg** provides PostgreSQL compatibility to replace SQLite.

Sensitive configuration. Sensitive environment variables (`SECRET_KEY`, `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, SMTP configuration) are externalised via **python-decouple**.

3.4 Backend Tests

Coverage scope. Tests cover the `users`, `events`, `registrations`, and `tags` modules.

Verified cases.

- role-based permission checks;
- CRUD operations;
- filtering logic;
- statistics computation;
- recommendation engine;
- notification sending.

Schema traceability. Migrations are generated and applied via Django's standard system (`makemigrations` / `migrate`), ensuring traceability of every schema evolution.

```
python manage.py test users events registrations tags
```

4 React Frontend

4.1 Application Architecture

4.1.1 Routing and Route Protection

General organisation. The application is a *Single Page Application* (SPA) whose entire navigation is handled client-side by **React Router v6**. The route tree is declared in `App.js` and covers **23 pages** split across four spaces: public, participant, organisation, and administrator.

Route protection.

- `ProtectedRoute` checks for a valid token via `isAuthed()` and redirects to `/login` on failure.
- `CompanyRoute` and `AdminRoute` additionally verify that the stored role matches `COMPANY` and `ADMIN` respectively.
- Any authenticated user with the wrong role is redirected to `/events`.
- All write routes in the organisation space (`/events/create`, `/events/:id/edit`, `/my-events`) are wrapped in `CompanyRoute`, and `/admin/*` routes in `AdminRoute`.
- Public pages (`/events`, `/events/:id`, `/company/:id`) remain accessible without authentication, with the display adapting based on the logged-in user's role.

4.1.2 Global State Management

Principle. The application uses neither Redux nor Zustand: state is divided into two independent layers, each persisted in `localStorage`.

`authStore.js`. It persists seven keys: `access_token`, `refresh_token`, `role`, `username`, `display_name`, `company_name`, and `user_id`. The centrepiece is the `getToken()` function, which includes expiration checking: it decodes the JWT's Base64 payload, reads the `exp` field, and clears the session if the token has expired, forcing a transparent re-login without server involvement.

```
export const getToken = () => {
  const token = localStorage.getItem("access_token");
  if (!token) return null;
  const payload = JSON.parse(atob(token.split(".")[1]));
  if (payload?.exp <= Math.floor(Date.now() / 1000)) {
    clearSessionStorage(); // clears all 7 keys
    return null;
  }
  return token;};
```

Derived helpers. The boolean helpers `isAuthed()`, `isCompany()`, `isAdmin()`, and `isParticipant()` all derive from `getToken()` and the stored role. They are used both in route guards and directly in components to adapt the display to the current role, without any additional API calls.

`PreferencesContext.` This global React context manages two persisted preferences:

- the **theme** (`light` / `dark`), automatically detected via `prefers-color-scheme` and manually overridable with a 560ms animated transition;
- the **locale** (`en` / `fr`), detected from `navigator.language`.

The `t(key)` function returns the translated string from a static dictionary and is accessible everywhere via the `usePreferences()` hook.

4.1.3 API Client

Responsibilities of `apiFetch()`.

- automatic injection of the `Authorization: Bearer <token>` header;
- base URL resolution from `REACT_APP_API_BASE`, allowing a switch between Django (port 8000) and Node.js (port 8001) without modifying any code;
- unified DRF error parsing covering the `detail` field, message arrays, and field-by-field validation dictionaries (`field_errors`, `non_field_errors`).

Call organisation.

- Calls are organised into six modules by business domain: `auth.js`, `events.js`, `registrations.js`, `companies.js`, `tags.js`, and `admin.js`.
- Each module exports named functions wrapping `apiFetch`, fully isolating transport logic from the rest of the interface.
- The `events.js` module also includes `deriveEventStatus()`, a utility function that computes the displayed status (*upcoming*, *live*, or *past*) client-side by comparing event dates with the current time, independently of the `status` field returned by the backend.

4.2 Key Pages and Demonstrable Features

4.2.1 Authentication

Login.

- The `/login` page offers a visual switch between the Participant form (email + password) and the Organisation form (alphanumeric identifier + password).
- Both flows call distinct endpoints and receive different JWT payloads: the participant token contains `email`, `first_name`, and `last_name`; the organisation token contains `company_name` and `company_identifier`.
- After login, the application reads the stored role and automatically redirects to `/dashboard` for an organisation or `/events` for a participant.

Registration and password.

- The `/register` page offers two distinct tabs.
- The organisation form includes a SIRET field that triggers automatic backend verification (SIRENE API) on submission.
- Password reset works in two steps: `ForgotPassword` submits the email address, then `ResetPasswordConfirm` consumes the signed link (`/reset-password/:uid/:token`) sent by email.

4.2.2 Event Search and Discovery

Entry point. The `/events` page is the main discovery entry point.

- The `SearchTopicInput` component detects the `#` prefix to display topic suggestions from the tag catalogue pre-fetched at application startup (`prefetchTags()` called in `App.js` on mount).
- Text search is validated by the `Enter` key to avoid spurious API calls during typing.
- The `/events/results` page displays results with six active filters: format (`ONSITE` / `ONLINE` / `HYBRID`), tags, start date, end date, city, and country.
- All active filters are encoded in the URL query parameters, allowing a filtered search to be shared or bookmarked, and enabling back/forward browser navigation.
- Removing a single filter is applied immediately without a page reload.

4.2.3 Event Detail and Registration – Many-to-Many Relationship in the UI

Principle. The `EventDetail` page is the most direct demonstration of the Many-to-Many relationship between participants and events. Its rendering adapts dynamically to the logged-in user's role.

- An **unauthenticated visitor** sees public information and a login button.
- A **logged-in participant** has a registration button whose label reflects the current status: *Register*, *Pending Validation*, *Waitlisted*, *Confirmed*, or *Cancel*.
- Duplicates are blocked at the UI level before even calling the API: if a registration already exists, the button changes state rather than allowing a second submission.
- The **owner organisation** has a side panel for managing registrations, displaying the registrant list, their statuses, and confirm / reject / remove actions.
- This panel uses an independent internal scroll so as not to affect the page's global scroll.
- The **administrator** has the same access, with a back link to `/admin/events` rather than `/my-events`.
- The visibility of the physical address and online link is also conditional: if the organiser has configured a reveal date, this information remains hidden until that date, in accordance with the `visible_address` and `visible_online` properties returned by the API.

4.2.4 Organisation Space

Dashboard.

- The dashboard (`/dashboard`) displays real-time metric cards: total views, confirmed registrations, pending registrations, average fill rate, and cancellation rate.
- Two export buttons trigger direct CSV file downloads: a global summary export and a per-event performance export.
- On desktop, the page runs in fixed viewport mode (`overflow: hidden` on the `body`) to confine scrolling to internal lists.

Event management.

- The `/my-events` page lists all of the organisation's events with coloured status badges computed client-side via `deriveEventStatus()` and quick actions (view, edit, delete).
- The event creation and editing form covers all model fields: title, description, banner, dates, capacity, format, registration mode (`AUTO` / `VALIDATION`), address and online link visibility with optional reveal date, and tags.
- Protection against unsaved navigation is active: a warning is displayed if the user attempts to leave the page with unsaved changes.
- This behaviour is shared with the `/profile/edit` page.

4.2.5 Profile Management

Public profiles.

- The **Participant profile** (`/participant/:id`) displays avatar, biography, field of interest, study level or job title according to type (`STUDENT` / `PROFESSIONAL`), tag badges, and GitHub / LinkedIn links; it is accessible to any authenticated user.
- The **Organisation profile** (`/company/:id`) shows logo, description, verification badge, social links, and published events; it is accessible without authentication.

Current profile. The `ProfileOverview` view (`/profile`) offers a concise presentation of the current profile, distinct from the editing form (`/profile/edit`), and more readable and less dense than the latter.

4.2.6 Administrator Space

Scope. The administrator space covers four pages.

- Participant management (`/admin/participants`) offers multi-field search, suspend / reactivate / delete actions, and a per-participant detail page.
- Organisation management (`/admin/companies`) presents in two columns organisations pending verification and those already verified, with the full workflow `PENDING` → `VERIFIED` / `REJECTED` / `NEEDS_REVIEW`.
- Event management (`/admin/events`) displays all statuses with format / status / period filters and deletion with a confirmation modal.
- The statistics dashboard (`/admin/statistics`) presents global platform indicators: users, events, and registrations by status.

4.3 Reusable Components and User Experience

Shared components. Several components are shared across pages and ensure visual and behavioural consistency throughout the interface.

- `AppHeader` is the main navigation bar. Adaptive to role and login state, hidden on authentication pages, its height is dynamically measured via a `ResizeObserver` and exposed as the CSS variable `-header-height`: each page can thus precisely compute its available height regardless of screen size.
- `NavUserMenu` is the contextual dropdown menu displaying the user's name and avatar (with initials fallback), along with Profile and Logout links, disabled for the administrator who has no profile page.
- `AppTopLinks` manages navigation tabs with a dynamically computed active tab. For organisations, it performs an asynchronous API call to check whether the current organisation owns the displayed event, and accounts for the `?context=my-events` URL parameter to maintain the active tab when navigating from `/my-events`.
- `SearchTopicInput` is the tag autocomplete search field: it detects the `#` prefix, displays matching suggestions, and manages topic selection / deselection as distinct tokens separate from free text.
- `PageShell` and `AuthPageShell` are the common layout templates ensuring consistent margins, maximum width, and positioning.

UX polish. Multilingual support (EN / FR) and light / dark theme, with automatic system preference detection, give the interface a level of polish well above a basic CRUD application, while illustrating React best practices: separation of concerns, reusable components, and clean side-effect management.

5 Node.js Backend and Comparative Analysis

5.1 Node.js Backend Overview

5.1.1 Stack and Structure

Technical stack.

- **Express.js** as the HTTP framework.
- **Sequelize** as the ORM, with SQLite support in development and PostgreSQL in production via the `pg` and `pg-hstore` drivers.
- `jsonwebtoken` for JWT generation and verification, and `bcryptjs` for password hashing.
- `Multer` for file uploads, `swagger-ui-express` for API documentation, `Nodemailer` for transactional emails, and `express-rate-limit` for request throttling.

MVC structure.

- The `models/` folder contains the main entities (`User`, `Event`, `Registration`, `Tag`, `BlacklistedToken`) as well as the join tables `UserTag` and `EventTag`.
- The `controllers/` concentrate business logic: `authController` (32K), `eventController` (30K), and `registrationController` (17K) are the largest.
- The `routes/` define endpoints for each domain (auth, events, registrations, tags, companies).
- The `middleware/` folder contains three files: `auth.js` for JWT verification, `permissions.js` for role-based access control, and `upload.js` for Multer.
- The `services/` group `emailService` (32K of HTML templates), `sireneService` for SIRET verification, and `tokenBlacklist` for token revocation.
- The `config/` folder contains `database.js` and `swagger.js` (35K lines of OpenAPI specification).
- Tests are in `tests/` with four files using Jest and Supertest.

5.1.2 Implemented Features

Principle. The Node.js backend is not a simplified implementation: it reproduces almost all of the Django backend's features.

- **Authentication**: participant and organisation registration (with SIRET verification via the SIRENE API), login, refresh token, logout with blacklist, password reset, and GDPR account deletion.
- **Event CRUD**: creation, editing, deletion, listing with filters (format, tags, dates, city, free text), per-event statistics, organisation dashboard, and tag-based recommendations.
- **Registration management**: capacity and deadline checking, waitlist with automatic promotion, organiser moderation, and CSV export.
- **Tags**: full CRUD.
- **Administration**: user management, organisation verification, platform statistics.
- **Transactional emails**: confirmation, waitlist, rejection, reminder, cancellation, organiser digest.
- **API documentation**: Swagger UI accessible at `/api/docs/`.

5.1.3 Key Implementation Differences

Despite equivalent functional coverage, several structural differences distinguish the two backends.

Models and associations. In Django, relationships (`ForeignKey`, `ManyToManyField`) are declared directly in the model and generate automatic *reverse lookups*. In Sequelize, each association must be explicitly defined in both directions in a central `index.js` file. For example, the relationship between `Registration` and `Event` requires two separate declarations:

```
// index.js -- Sequelize associations (both directions required)
Registration.belongsTo(Event, { foreignKey: 'event_id', as: 'event' });
Event.hasMany(Registration, { foreignKey: 'event_id', as: 'registrations' });
```

In Django, the same relationship is declared in a single line via a `ForeignKey` and the reverse lookup is generated automatically.

Permissions. Django REST Framework provides reusable declarative classes (`IsAuthenticated`, `IsAdminUser`) and allows custom permissions to be created. In Express, we wrote individual middleware functions. Here is an example from `permissions.js`:

```
function requireCompany(req, res, next) {
  if (!req.user)
    return res.status(401).json({ error: 'Authentication required.' });
  if (req.user.role !== 'COMPANY')
    return res.status(403).json({ error: 'Reserved for organisers.' });
  next();
}
```

Five such middlewares were written: `requireParticipant`, `requireCompany`, `requireAdmin`, `requireCompanyOrAdmin`, and `requireAuthenticated`. On the Django side, these checks are declarative and require no manual validation code.

Authentication. Django simplifies JWT with `djangorestframework-simplejwt`, which requires only a few lines of configuration in `settings.py`. In Node.js, the `auth.js` middleware manually implements the entire flow: Bearer token extraction, blacklist check, JWT decoding, database user lookup, and active account verification. A second middleware (`optionalAuthenticate`) handles public routes that benefit from additional information when a user is logged in.

Registration uniqueness constraint. Duplicate prevention illustrates the difference in approach well. In Sequelize, the constraint is defined at the model level:

```
// Registration.js -- Sequelize uniqueness constraint
indexes: [{
  unique: true,
  fields: ['participant_id', 'event_id'],
  name: 'unique_participant_event',
}]
```

In Django, the equivalent is a simple `unique_together` in the `Meta` class, complemented by automatic serializer validation. In Node.js, this check must also be performed manually in the controller before creation.

Database configuration. The `database.js` file handles three scenarios automatically: connection via `DATABASE_URL` if present (PostgreSQL with SSL); separate variables (`DB_HOST`, `DB_NAME`, etc.) for manual configuration; SQLite by default in local development. Django centralises this logic in `settings.py` in a more integrated manner thanks to `dj-database-url`.

5.2 Detailed Comparative Analysis: Django vs Node.js

The following comparison is based on our concrete development experience of both backends for the same application. Each criterion reflects differences actually observed during the project.

TABLE 4. Detailed comparison: Django + DRF vs Node.js + Express, based on the Neurovent implementation.

Criterion	**Django + DRF**	**Node.js + Express**
Project structure	Strict, enforced by the framework (apps, serializers, views)	Fully manual (`controllers/`, `routes/`, `models/`)
ORM	Django ORM: auto-migrations, expressive queries, reverse lookups	Sequelize: explicit associations in both directions, more verbose
Authentication	`simplejwt`: configuration in a few lines	Manual: bcrypt + jsonwebtoken + custom middleware + blacklist
Permissions	Reusable declarative classes (`IsCompany`, `IsParticipant`)	Per-role middleware functions (`requireCompany`, `requireAdmin`...)
Validation	Serializers with built-in validation and automatic error messages	`express-validator` + manual checks in controllers
Error handling	Structured JSON responses automatically handled by DRF	Manual try-catch + global error handler in `app.js`
API documentation	`drf-spectacular`: auto-generated from code	`swagger-jsdoc`: 35K lines of manual specification
Tests	Built-in framework (`manage.py test`)	Jest + Supertest, mocks for emails and SIRENE
Emails	Built-in Django email templates	Nodemailer with manual HTML templates (32K in `emailService.js`)
Built-in admin	Yes (Django Admin)	No (custom admin routes)
Code volume	More concise thanks to abstractions	Larger: `authController` 32K, `eventController` 30K
Deployment	gunicorn + WhiteNoise, well documented	`render.yaml` preconfigured, but issues with `pg`, CORS, SSL

5.3 Summary: When to Choose Which?

Django. The right choice when the project involves complex business rules (waitlist, role-based permissions, strict validation), a need for out-of-the-box administration, or when productivity is the priority. Built-in tools (ORM, serializers, admin, automatic migrations) reduce code volume and risk of errors.

Node.js. Better suited to projects requiring real-time functionality (WebSocket, streaming), maximum architectural flexibility, or a full-JavaScript stack. The manual approach provides a deeper understanding of the internal mechanisms of authentication, permissions, and database configuration.

In Neurovent. Django remains the reference backend due to the complexity of the data model and the numerous business rules (waitlist, SIRET verification, exports, transactional emails). The Node.js backend, developed as an alternative implementation, nonetheless provided a better understanding of what Django's abstractions conceal: each layer (authentication, permissions, validation, error handling) had to be implemented manually, which constitutes a significant pedagogical contribution.

6 Deployment

6.1 Deployment Platforms

Principle. Neurovent's deployment relies on two complementary cloud platforms.

Vercel. Specialised in hosting frontend applications and static sites. It is particularly well-suited to *Single Page Applications* built with React, Next.js, or Vite. Its main advantages are automatic deployment on every Git push, global CDN file distribution, native environment variable management, and automatic HTTPS provisioning. For Neurovent, Vercel hosts the **React frontend**.

Render. A versatile cloud platform supporting web servers, background workers, and managed databases. It natively supports Python, Node.js, and PostgreSQL, with automatic deployment from GitHub and real-time log monitoring. For Neurovent, Render hosts the **Django backend** (main API), the **Node.js backend** (comparative implementation), and the **PostgreSQL databases** used by each.

Justification for the separation. The choice to separate the frontend from the backends is explained by the difference in the nature of the workloads. The frontend produces static files that benefit from a CDN; the backends execute dynamic code and communicate with a database. This separation allows each service to be scaled independently and ensures that a backend failure does not prevent the frontend from loading.

6.2 Frontend Deployment on Vercel

Vercel configuration.

- The React frontend is deployed on Vercel under the project name `neurovent-web`, with `frontend-react` as the root directory.
- The preset is configured for Create React App: the build command is `npm run build`, the output directory is `build`, and installation is done via `npm install`.
- The only frontend environment variable is `REACT_APP_API_BASE`, pointing to the Django backend on Render.
- It is used in `client.js` as the base URL for all API calls:

```
// client.js -- base URL configurable via environment variable
const API_BASE = process.env.REACT_APP_API_BASE
                 || "http://localhost:8000";
```

Important note. Create React App injects environment variables at **build time**, not at runtime. Any change to `REACT_APP_API_BASE` therefore requires a full frontend redeployment to take effect. Every push to the `main` branch automatically triggers a new build and deployment on Vercel.

6.3 Django Backend Deployment on Render

Service parameters. The main Django backend is deployed on Render as a Web Service named `neurovent-api`, in the Frankfurt (EU Central) region, on the Free plan, from the `main` branch and the `backend-django` root directory. The build command chains three steps:

```
pip install -r requirements.txt \
  && python manage.py collectstatic --noinput \
  && python manage.py migrate
```

Start command. The service is then started with `gunicorn config.wsgi:application`.

Required adaptations.

- Django's development server (`runserver`) was replaced by **gunicorn**, a production-grade WSGI server.
- **WhiteNoise** was integrated as middleware to serve static files directly from `staticfiles/` with automatic compression and cache-busting, eliminating the need for a separate web server like Nginx.
- The database connection is managed via `DATABASE_URL`, parsed by `dj-database-url` with mandatory SSL and connection pooling (`conn_max_age=600`).
- Cross-origin requests are handled by `django-cors-headers`, whose `CORS_ALLOWED_ORIGINS` variable is configured with the Vercel frontend URL.
- For emails, `settings.py` automatically detects whether SMTP credentials are complete and falls back to the console backend in their absence, avoiding crashes.
- All critical variables (`SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, `FRONTEND_URL`) are read from environment variables via `python-decouple`.

6.3.1 PostgreSQL Database on Render

The database is a PostgreSQL instance named `neurovent-db`, hosted in the Frankfurt region under the Free plan. The configuration in `settings.py` automatically selects the engine based on the presence of `DATABASE_URL`:

```
DATABASE_URL = config('DATABASE_URL', default='')

if DATABASE_URL:
    DATABASES = {
        'default': dj_database_url.parse(
            DATABASE_URL, conn_max_age=600,
            ssl_require=True)
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
```

Operational constraints.

- The free database expires on **6 May 2026** unless upgraded.
- The backend uses the **Internal Database URL** for the production connection.
- The **External Database URL** is only used from a local machine, for example to inject demo data.
- The free tier backend enters sleep mode after inactivity, and the first request after waking may take 30 to 60 seconds.

6.3.2 Demo Data Injection

Context. After an initial deployment, the database is empty: migrations create tables but not content.

Procedure. To populate the database with demo data from a local machine:

```
cd backend-django
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
export DATABASE_URL='<External Database URL from Render>'
export FRONTEND_URL='https://<vercel-production-url>'
python scripts/reset_and_seed_demo.py
```

Note. The External Database URL must be used here because the command is executed from a local machine, outside Render's internal network.

6.4 Node.js Backend Deployment on Render (Comparative)

Principle. The Node.js backend was deployed separately on Render with its own PostgreSQL database for the comparative study. The project includes a `render.yaml` file that declaratively defines the infrastructure:

```
services:
  - type: web
    name: neurovent-backend
    env: node
    buildCommand: npm install
    startCommand: node server.js
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: neurovent-db
          property: connectionString
      - key: JWT_SECRET
        generateValue: true
databases:
  - name: neurovent-db
    databaseName: neurovent
    user: neurovent
```

Application configuration.

- The `database.js` configuration handles three scenarios automatically: connection via `DATABASE_URL` if present (PostgreSQL with SSL); separate variables (`DB_HOST`, `DB_NAME`, etc.) for manual configuration; SQLite by default in local development.
- The CORS configuration in `app.js` dynamically allows all `*.vercel.app` subdomains via a regex:

```
// app.js -- dynamic CORS for Vercel
if (/^https:\/\/[^.]+\.vercel\.app$/.test(origin))
  return callback(null, true);
```

Initialisation. On startup, `server.js` automatically creates an administrator account if none exists, using the `ADMIN_EMAIL` and `ADMIN_PASSWORD` variables.

6.5 Deployment Architecture

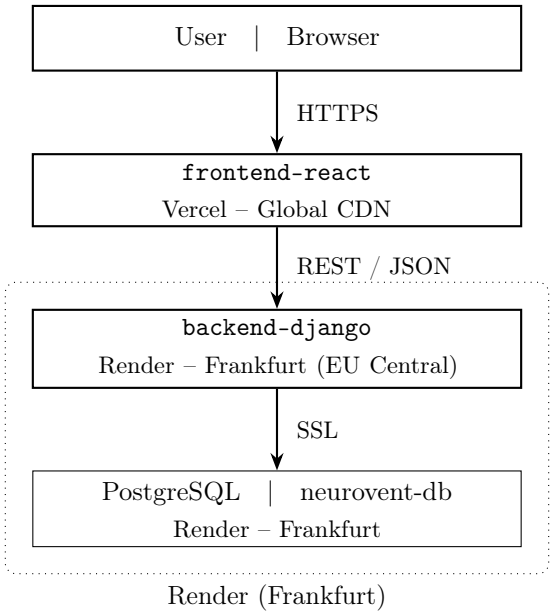

FIGURE 2. Production deployment architecture of Neurovent.

Production flow.

1. the user accesses the site via the Vercel URL;
2. the browser loads the React SPA from the CDN;
3. React sends HTTP requests to `https://neurovent-api.onrender.com` via the `REACT_APP_API_BASE` variable;
4. Django verifies the JWT token, processes the request, queries PostgreSQL, and returns a JSON response that React renders.

6.6 Environment Variables

TABLE 5. Production environment variables.

Variable	Service	Role
`REACT_APP_API_BASE`	Vercel	URL of the Django backend on Render
`DATABASE_URL`	Render	Internal PostgreSQL connection string
`SECRET_KEY`	Render	Django secret key for token signing
`DEBUG`	Render	`False` in production
`ALLOWED_HOSTS`	Render	`.onrender.com`, `localhost`, `127.0.0.1`
`CORS_ALLOWED_ORIGINS`	Render	Vercel frontend URL
`FRONTEND_URL`	Render	Vercel URL (email links, password reset)

6.7 Deployment Challenges Encountered

6.7.1 Django Backend

Overview. The Django backend deployment was relatively smooth thanks to the preparation work done in `settings.py`.

Main adaptations.

- adding `gunicorn`, `whitenoise`, `dj-database-url`, and `psycopg[binary]` to `requirements.txt`;
- externalising all critical parameters via environment variables;
- integrating an email fallback mechanism to avoid crashes when SMTP is not configured.

6.7.2 Node.js Backend

Overview. The Node.js backend deployment encountered several concrete problems, each requiring specific investigation.

- The first deployment failed with the error `"Please install pg package manually"`: the `pg` package was not in the initial dependencies since local development used SQLite.
- After correction, the backend crashed with a `ConnectionRefusedError` because the database environment variables had not been configured on Render.
- A third failure appeared with the error `"database neurovent_db does not exist"`: the manually configured name did not match the actual name created by Render.
- Finally, the Sequelize configuration required explicit SSL options (`ssl: { require: true, rejectUnauthorized: false }`) for the PostgreSQL connection to work.

6.7.3 Frontend – Backend Connection

Two main issues were encountered.

- **CORS blocking**: the browser was rejecting requests with the message `"No Access-Control-Allow-Origin header"`. On the Django side, the `CORS_ALLOWED_ORIGINS` variable had to include the Vercel domain. On the Node.js side, the CORS regex in `app.js` was adjusted to accept all `*.vercel.app` subdomains.
- **Wrong backend URL on the frontend**: even after configuring CORS, the frontend kept calling `localhost:8000`. The `REACT_APP_API_BASE` variable had been added on Vercel, but the frontend had not been redeployed. Since Create React App injects variables at build time, without a redeployment the old default value remained in use.

6.8 Current Deployment Status

System state.

- The **frontend** is accessible via Vercel with automatic HTTPS.
- The **Django backend** is running at `https://neurovent-api.onrender.com`, with API documentation available at `/api/docs/`.
- The **database** PostgreSQL is hosted on Render under the free plan, with an expiry date of **6 May 2026**.
- The **Node.js backend** is deployed separately on Render with its own PostgreSQL instance, serving as the comparative implementation.

Current limitations.

- the backend enters sleep mode after inactivity (wake-up in 30 to 60 seconds);
- the database has an expiry date;
- media files (avatars, banners) use local storage rather than an external service such as AWS S3;
- actual email sending requires a complete SMTP configuration, without which emails are only displayed in the console.

7 Conclusion

7.1 Project Summary

Compliance with the requirements.

- The `Registration` model implements the central Many-to-Many relationship with five statuses, a uniqueness constraint, and a waitlist logic with automatic promotion.
- Authentication relies on JWT with three distinct roles (`PARTICIPANT`, `COMPANY`, `ADMIN`), each with granular permissions and a dedicated functional space.
- Event filtering combines six criteria on both the API and frontend.
- Deployment is live in production, with the frontend accessible on Vercel and the Django backend running on Render with a PostgreSQL database.

Features beyond the requirements.

- SIRET verification via the SIRENE API;
- CSV exports for organisers;
- deferred visibility for addresses and links;
- transactional emails (confirmation, waitlist, reminders, digest);
- auto-generated OpenAPI documentation;
- multilingual support (EN / FR) and light / dark theme with automatic system preference detection.

7.2 Overall Assessment

Neurovent grew beyond a simple course exercise into a genuinely complete product. Each feature raised new requirements: a registration system needs a waitlist, a waitlist needs automatic promotion, automatic promotion needs transactional emails. Addressing each of them coherently gave the project a level of internal consistency that reflects the complexity of a real-world application.

The two-backend approach was ultimately the most instructive part of the exercise, making the trade-offs between Django and Node.js concrete in a way that no theoretical comparison could.

7.3 What We Learned

Key lessons.

- This project allowed us to confront concepts studied in class with concrete integration and deployment challenges.
- The React / Django full-stack integration taught us to manage the complete flow of an authenticated request: from storing the JWT in `localStorage` to server-side verification, through CORS error handling and silent token renewal.
- Designing the `Registration` model showed us that a Many-to-Many relationship with embedded business logic (waitlist, automatic promotion, uniqueness constraint) requires thinking that goes well beyond a simple join table.
- The empirical comparison between Django and Node.js, carried out over an identical functional scope, was one of the most formative aspects of the project.
- Manually implementing in Express what Django provides through its abstractions (serializers, declarative permissions, ORM with automatic migrations) gave us a deeper understanding of what each layer of a framework actually does.
- Finally, production deployment confronted us with problems absent in local development: SSL configuration for PostgreSQL, environment variable management between Vercel and Render, CORS between different domains, and the fact that Create React App injects variables at build time rather than runtime.
- These difficulties, documented in Section 6, constitute tangible learning about the web application lifecycle beyond development.

7.4 Future Directions

- Integrating **WebSocket** via Django Channels would enable real-time notifications on registration status changes, replacing the current model of periodic requests.
- A port to **React Native** could reuse the existing API to offer a mobile application without backend redevelopment.
- Moving to a **PostgreSQL database on a durable plan** (paid) and adding external media storage (AWS S3 or equivalent) would make the architecture viable for real production use.
- Finally, setting up **automated end-to-end tests** via Playwright — whose configuration already exists in the repo — would complete the project's validation coverage.

Appendices

A – Simplified Entity-Relationship Diagram

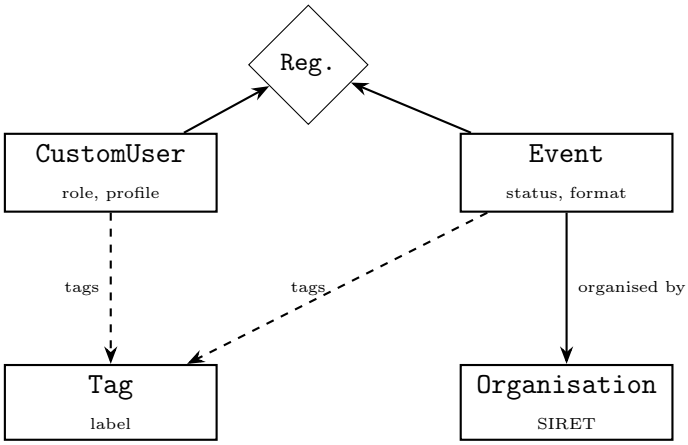

FIGURE 3. Simplified entity-relationship diagram.

B – Local Installation Instructions

```
# Django backend
cd backend-django
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver # http://localhost:8000

# Node.js backend (optional)
cd backend-node
npm install
npm run dev # http://localhost:8001

# React frontend
cd frontend-react
npm install
npm start # http://localhost:3000
```

C – Main Endpoints Summary

Main endpoints of the Django API (reference backend).

Endpoint	Method	Description
`/api/auth/register/participant/`	POST	Participant registration
`/api/auth/register/company/`	POST	Organisation registration
`/api/auth/login/participant/`	POST	Participant login → JWT
`/api/auth/login/company/`	POST	Organisation login → JWT
`/api/auth/me/`	GET/PATCH/DELETE	Current profile
`/api/events/`	GET	List with filters
`/api/events/create/`	POST	Create an event
`/api/events/:id/`	GET	Event detail
`/api/events/:id/stats/`	GET	Registration statistics
`/api/registrations/`	POST	Register for an event
`/api/registrations/my/`	GET	My registrations
`/api/registrations/event/:id/`	GET	Event registrants (owner)
`/api/auth/admin/users/`	GET	User list (admin)
`/api/docs/`	GET	Interactive documentation (Swagger)

D – Application Screenshots

Authentication

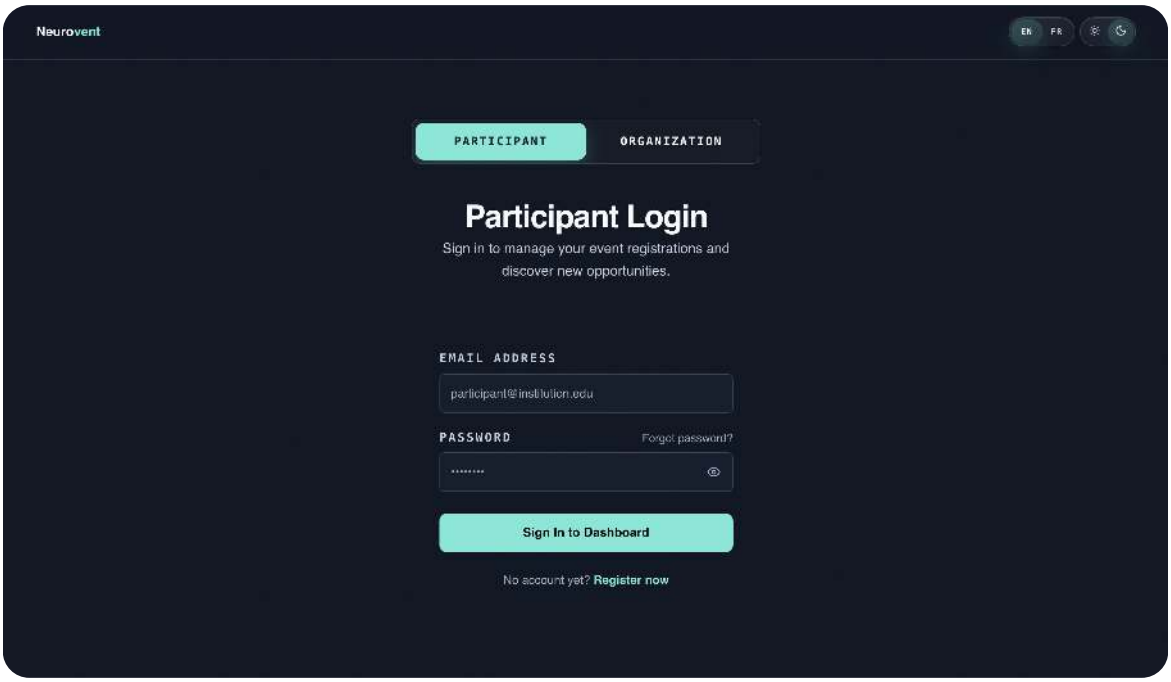

FIGURE 4. Participant login page.

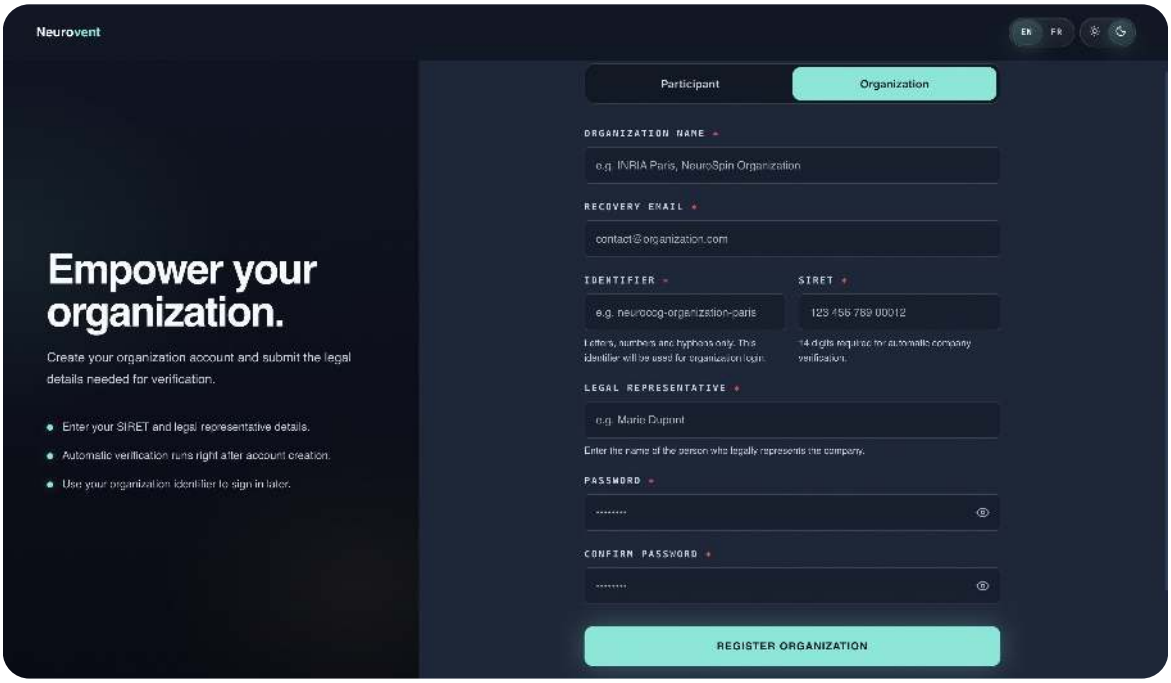

FIGURE 5. Organisation registration form.

Participant Space

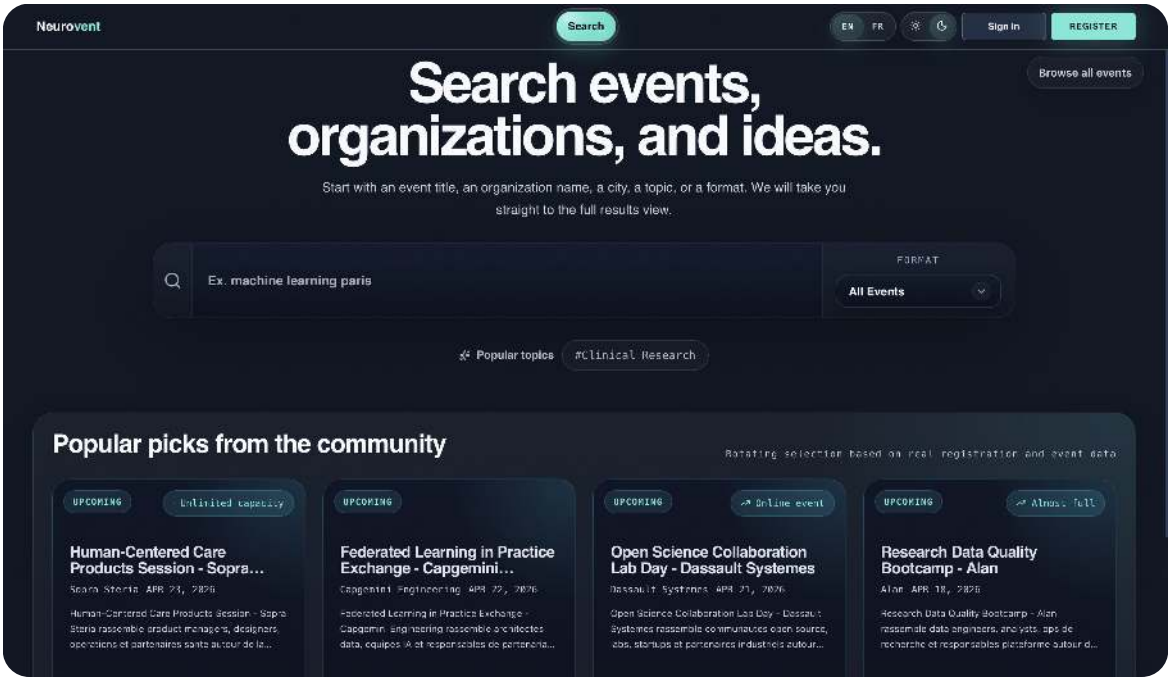

FIGURE 6. Event search page.

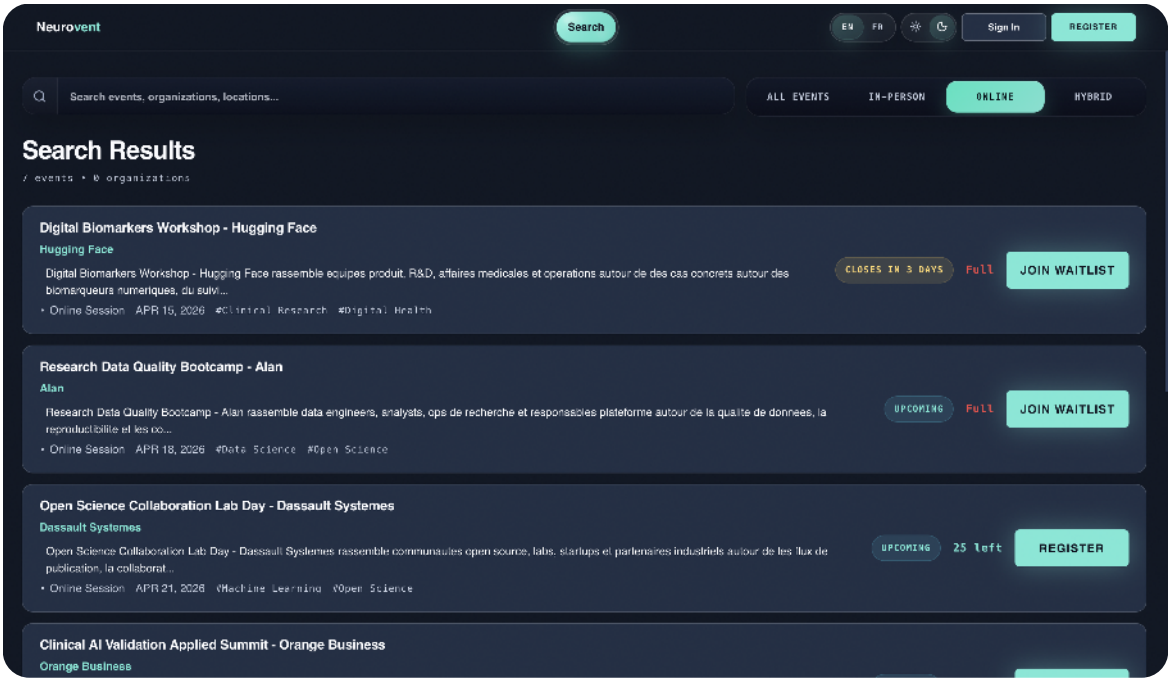

FIGURE 7. Event search results.

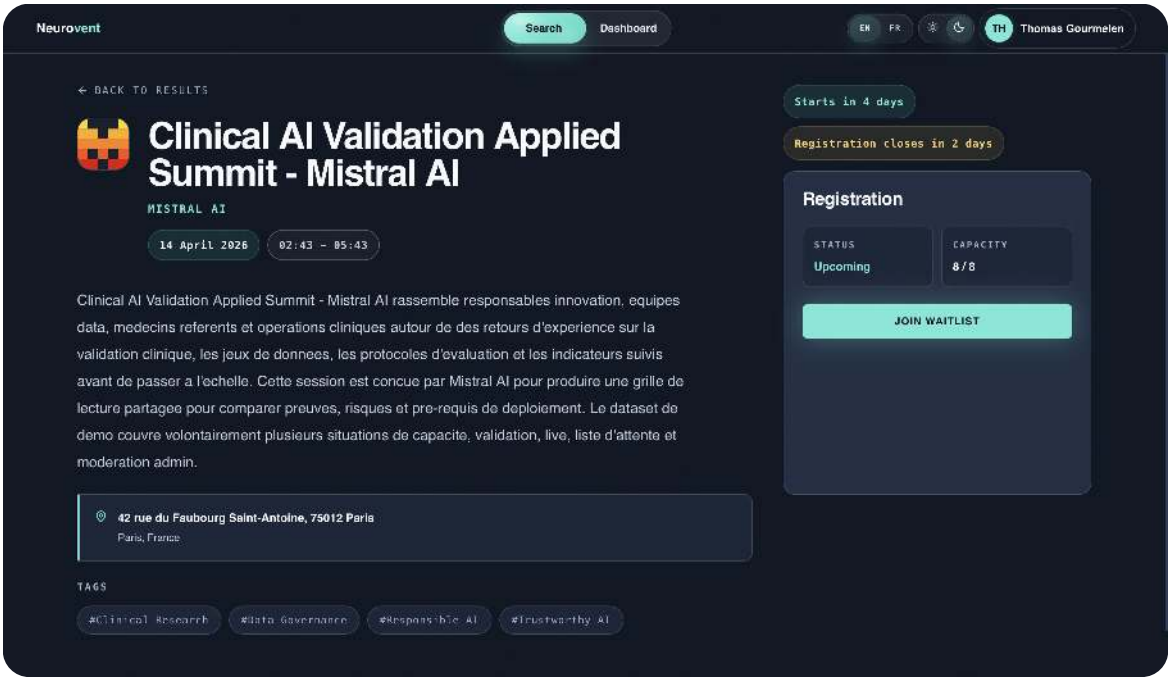

FIGURE 8. Event detail page with registration button.

Organisation Space

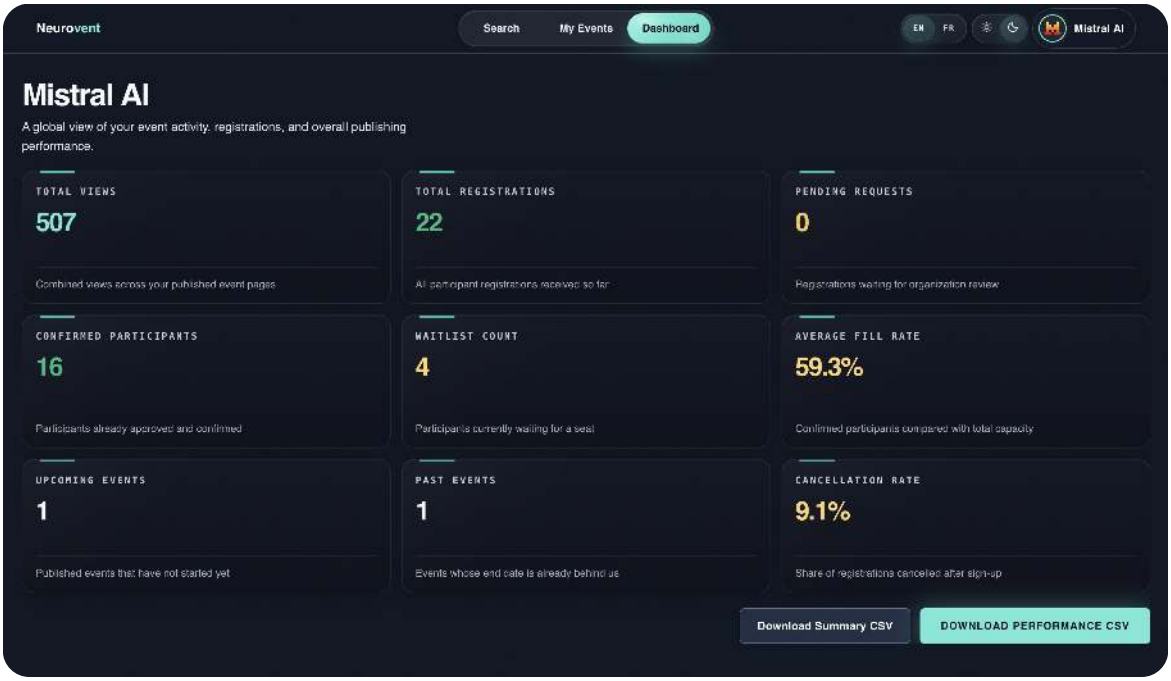

FIGURE 9. Organisation dashboard with metric cards and CSV exports.

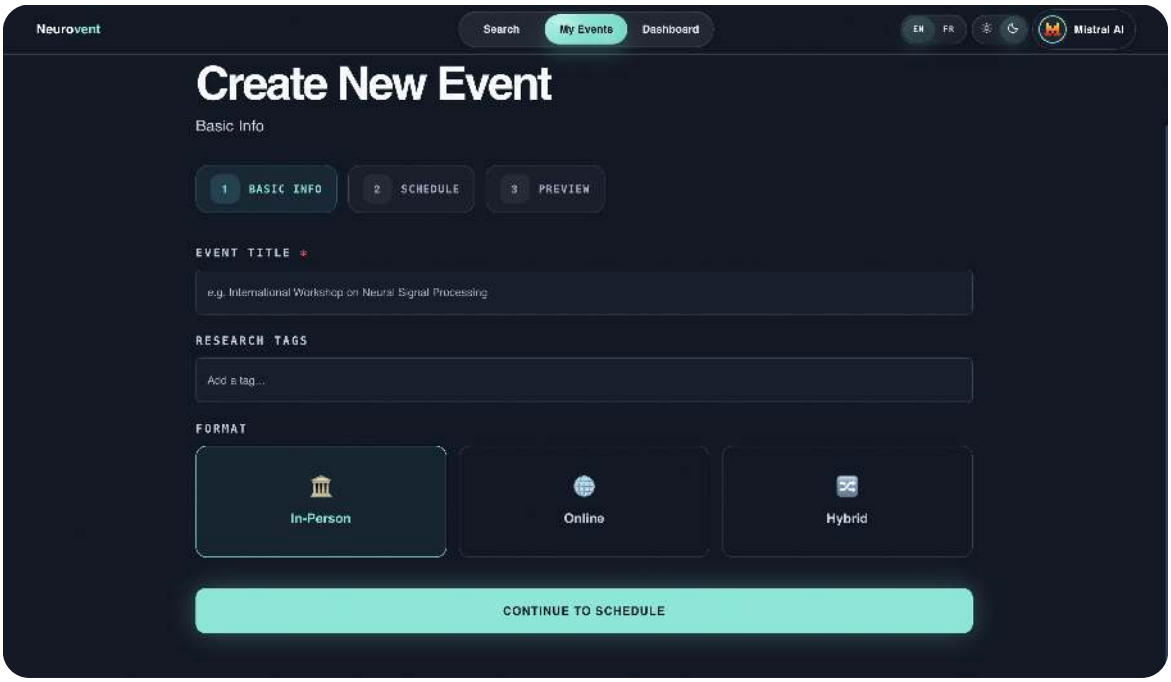

FIGURE 10. Event creation form.

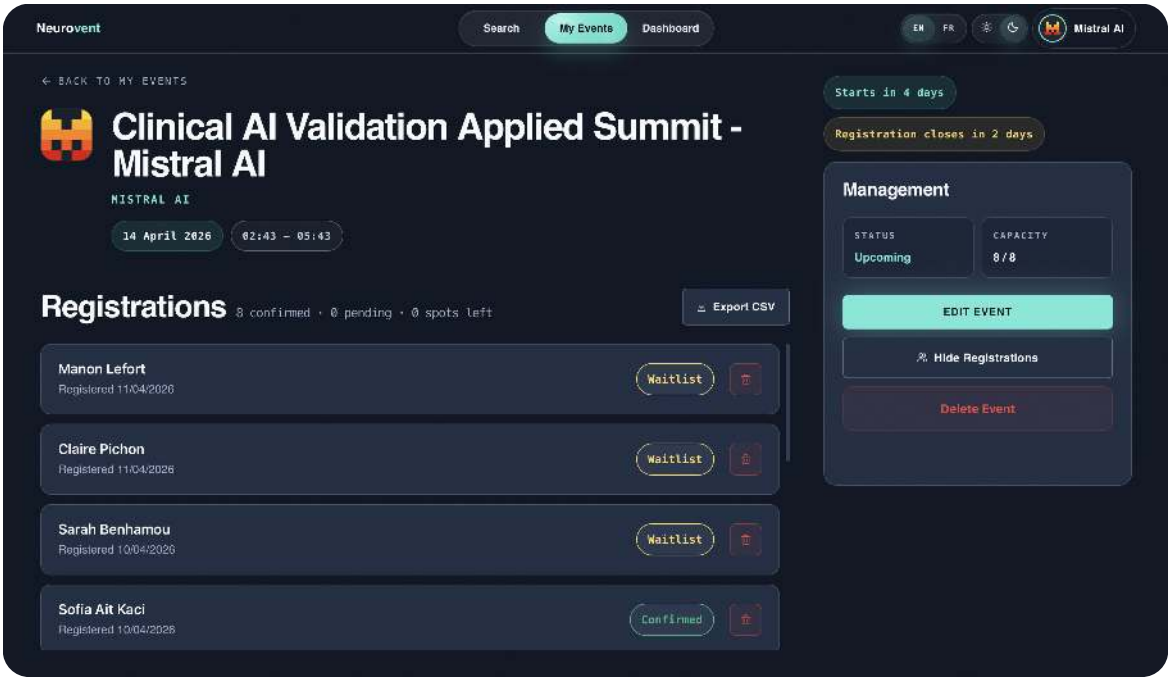

FIGURE 11. Registration management panel (organisation owner view).

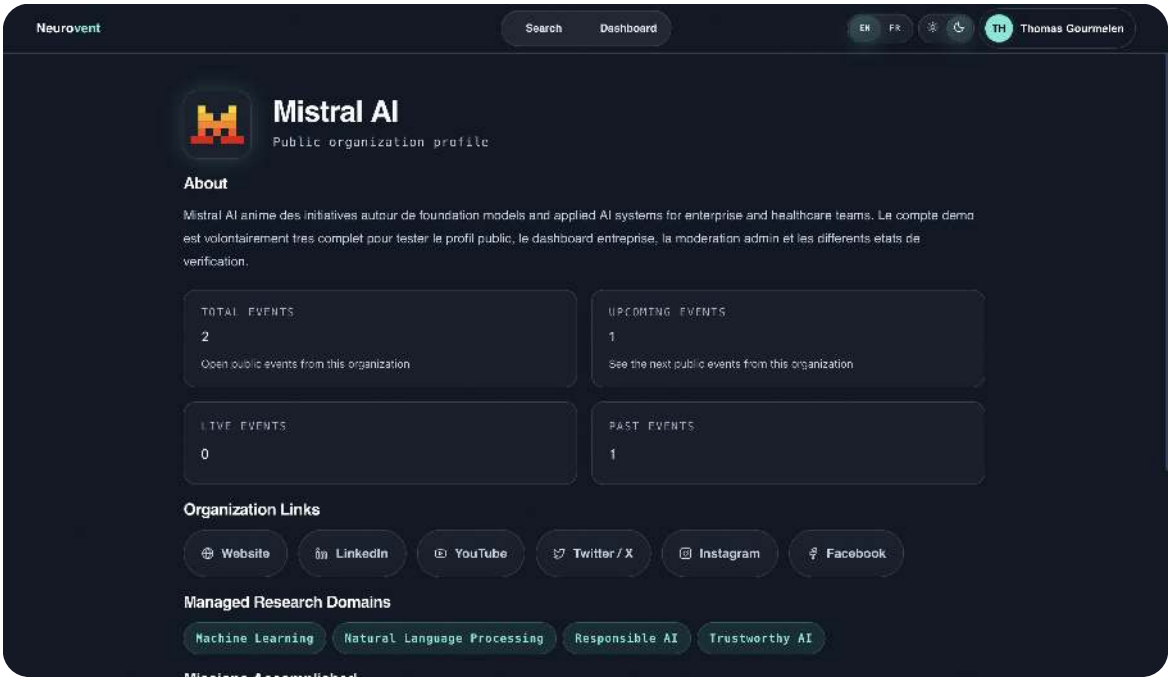

FIGURE 12. Organisation public profile.

Administrator Space

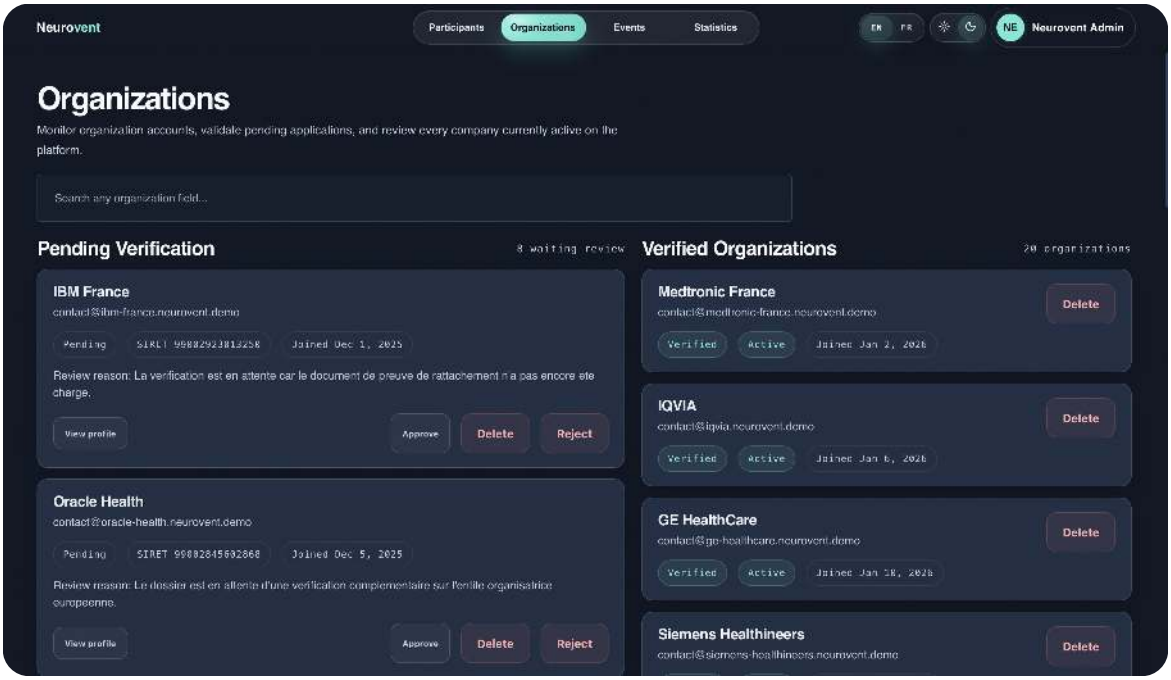

FIGURE 13. Admin – organisation list with verification status.

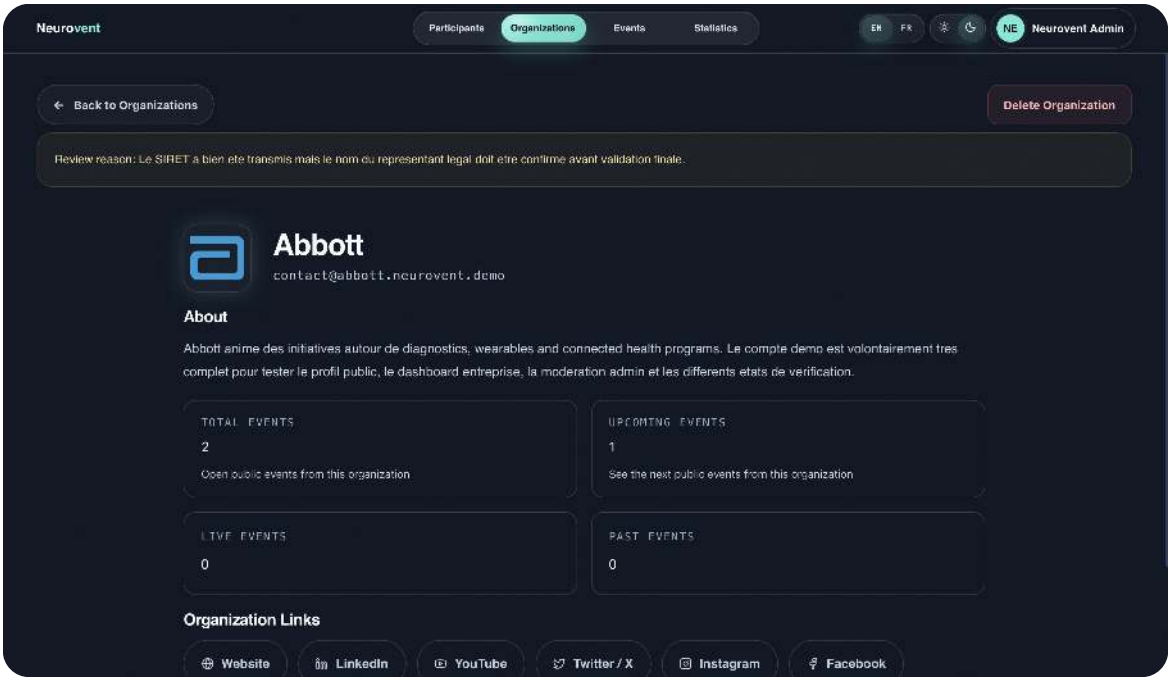

FIGURE 14. Admin – pending organisation verification detail.

Responsive Design

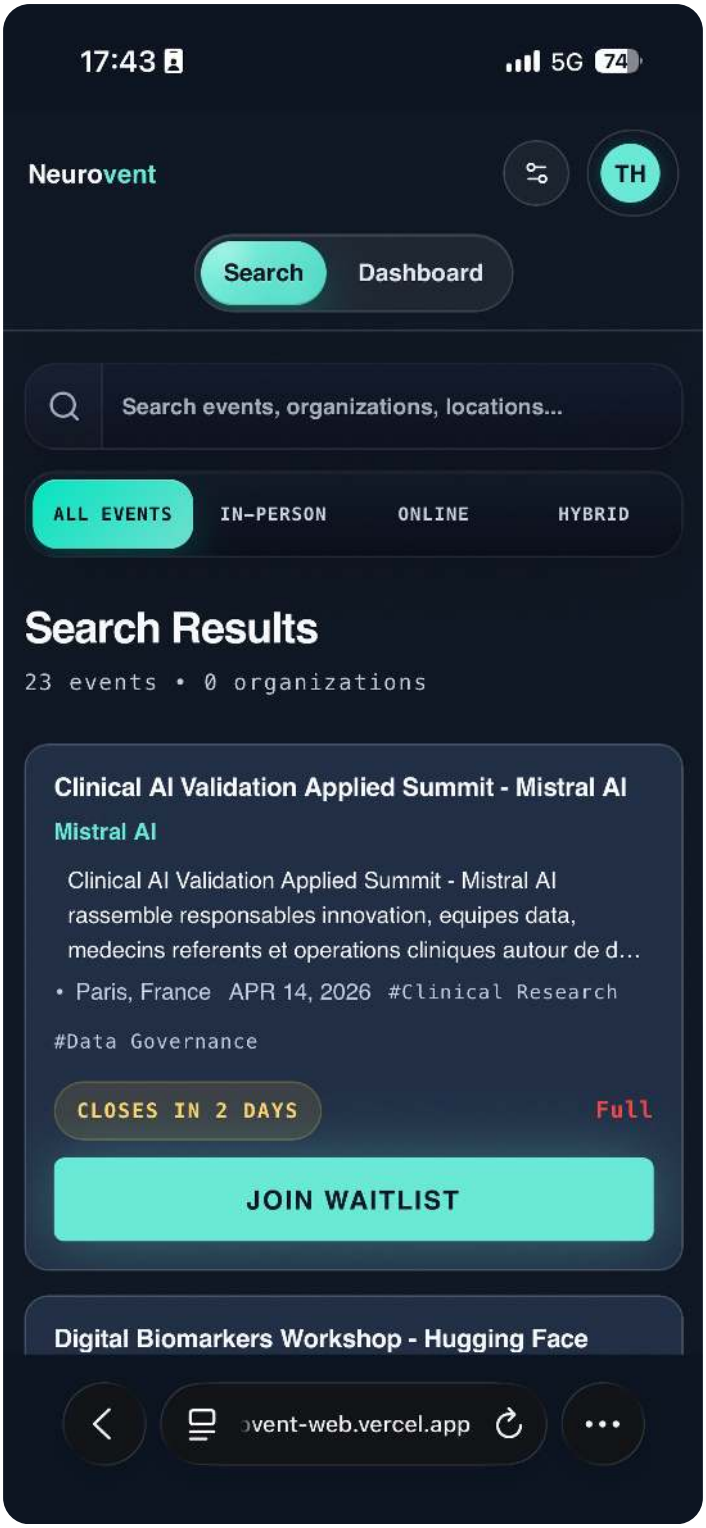

FIGURE 15. Mobile view of the application.

Note on the Use of Artificial Intelligence

For the writing of this report, we used the artificial intelligence model **Claude (Anthropic)**. Its use was limited to the following points:

- **Rephrasing and proofreading:** improving the clarity of certain formulations and correcting spelling or syntax errors.
- **Formatting:** assistance with content structuring and the writing of technical passages in English.

All analyses, technical decisions, implementations, and conclusions presented in this document are the result of the authors' own work. All intellectual property rights of this document remain exclusively attributed to its authors.
